

MLib Validation & Price Checking

Nicholas Burgess



MHI

Nicholas.Burgess@uk.mizuho-sc.com

18th March 2016

The Pricing Problem

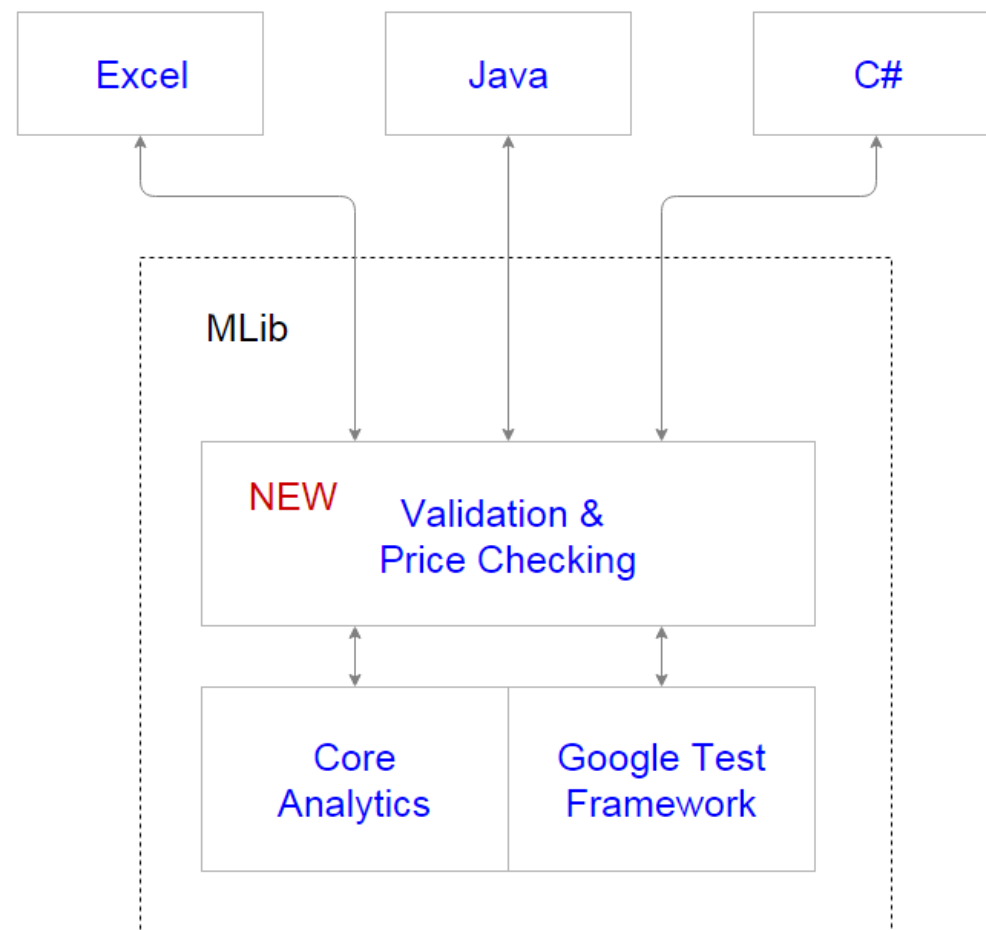
- Price test coverage is low & presents a significant risk
- Validation & price testing needs to be tackled in a organized manner
- Currently validation checks are not robust enough and fragmented

Price Testing Motivation

- Price testing reduces risk & necessary for “One Mizuho” code sharing
- Validation logic should not be in Excel or other client APIs
- Additional client price testing is most welcome but should not be a replacement for MLib internal price tests

Validation API

- Centralized Validation
- Centralized Price Tests
- Centralized Exception Handling
- Prevents Test Duplication
- API Synchronization



Project: **VANILLA_ADDIN**

- Function Signature: `mirSetUpOISCurve`

Project: **validation_api**

- Prefix "**try**" to the Excel function signature
- Filename: `tryMirSetUpSwapCurve.cpp`
- Function Signature: `tryMirSetUpSwapCurve`

Example Function Signatures

Excel Add-In NameSpace + Function Namespace + Function Signature

Project: ETRADING_ADDIN

meSwapPV	me + Swap + PV
meSwapCashflows	me + Swap + Cashflows
meSwapParRate	me + Swap + ParRate
meSwapPV01	me + Swap + PV01
meSwapDV01	me + Swap + DV01

Project: validation_api

- Prefix "**try**" to the Excel function signature
- Filename: tryMeSwapPV.cpp
- Function Signature: tryMirSetUpSwapCurve

```
1 // Excel API
2 string mirSetUpSwapCurve( excelArg1 , excelArg2 , ... )
3 {
4     // Marshal arguments from Excel Types to C++ Types
5     cppArg1 = marshal( excelArg1 );
6     cppArg2 = marshal( excelArg2 );
7     ...
8
9     // Call Function in Validation Layer
10    return tryMirSetUpSwapCurve( cppArg1 , cppArg2 , ... );
11 };
```

```
1 // Validation / Price Check API
2
3 // tryMirSetUpSwapCurve.cpp
4 string tryMirSetUpSwapCurve( cppArg1, cppArg2, ... )
5 {
6     // 1. Add Structured & Other Exception Handling
7     // 2. Check All Arguments are Valid
8     // 3. Function Testing, Recording & Logging
9
10    If ( TestsPassed == true )
11    {
12        // Call Underlying Function(s)
13        return calibrateCurve( cppArg1, cppArg2, ... );
14    }
15    else
16    {
17        // Throw an Error
18        THROW_ERROR( Error Message );
19    }
20 };
```

The End